isec
Politécnico de Coimbra

Licenciatura em Engenharia Informática – 2022/23

# Programação

## 6: Recursividade
## Árvores Binárias

Francisco Pereira (xico@isec.pt)

# Funções Recursivas

isec
Politécnico de Coimbra

- Há problemas que são definidos naturalmente de forma recursiva

- Nestes casos, pode ser mais simples escrever funções recursivas
  - Funções que se chamam a si próprias

# Exemplo

- Calcular o fatorial de um inteiro positivo

```
0! = 1

1! = 1

n! = n × (n-1)!, se n > 1
```

Definição recursiva

# Função Fatorial

Condição de paragem

```
int fatorial(int val)
{
   if(val <= 1)
       return 1;
   else
       return val * fatorial(val-1);
}
```

Chamada recursiva

# Recursividade: Programação Preguiçosa

isec
Politécnico de Coimbra

- A tarefa a resolver é demasiado complicada
  - Qual é a situação mais simples de todas?
  - Como é que a tarefa complicada pode ser ligeiramente simplificada?

# Estrutura de uma função recursiva

isec
Politécnico de Coimbra

- Condição de paragem / Caso de base
  - Termina as chamadas recursivas
  - Surge antes da chamada recursiva
- Chamada recursiva
  - A tarefa a resolver na nova chamada deve ser mais simples
  - O caminho percorrido pelas chamadas recursivas termina na condição de paragem

# Função recursivas incorretas

```
int fatorial(int val)
{
   return val * fatorial(val-1);
   if(val <= 1) return 1;
}
```

```
int fatorial(int val)
{
   if(val <= 1) return 1;
   else return val * fatorial(val);
}
```

```
int fatorial(int val)
{
   if(val <= 1) return 1;
   else return val * fatorial(val+1);
}
```

# Vantagens e desvantagens

isec
Politécnico de Coimbra

- Vantagens
  - Código mais compacto
  - Fácil de escrever e de compreender
- Desvantagens
  - Ocupação intensiva de recursos (pilha do programa)

# Alguns Exemplos

isec
Politécnico de Coimbra

- Manipulação de strings
- Operações sobre listas ligadas
- Manipulação de vetores
- Puzzle das Torres de Hanói
- Árvores binárias

# Exemplo 1: Contar

- Calcular o tamanho de uma string

```
int conta(char *st)
{
   if(*st == '\0')
       return 0;
   else
       return 1+conta(st+1);
}
```

# Exemplo 2: Inverter

- Escrever uma string por ordem inversa

```
void puts_inv(char *st)
{
   if(*st == '\0')
       return;
   else
   {
       puts_inv(st+1);
       putchar(*st);
   }
}
```

# Exemplo 3: Capicua

- Verificar se uma string é capicua

```
int capicua(char *st, int tam)
{
   if(tam <= 1)
       return 1;
   else if(*st != *(st+tam-1))
       return 0;
   else
       return capicua(st+1, tam-2);
}
```

2 condições de paragem

# Manipulação de listas ligadas

- Operações a implementar:
  1. Contar o número de nós
  2. Escrever a informação por ordem inversa

```
typedef struct numero no, *pno;

struct numero {
   int val;
   pno prox;
};
```

# Operação 1: Contar

isec
Politécnico de Coimbra

```
int conta_nos(pno p)
{
   if(p == NULL)
       return 0;
   else
       return 1 + conta_nos(p->prox);
}
```

# Operação 2: Inverter

```
void escreve_inv(pno p)
{
   if(p == NULL)
       return;
   else
   {
       escreve_inv(p->prox);
       printf("%d\t", p->val);
   }
}
```

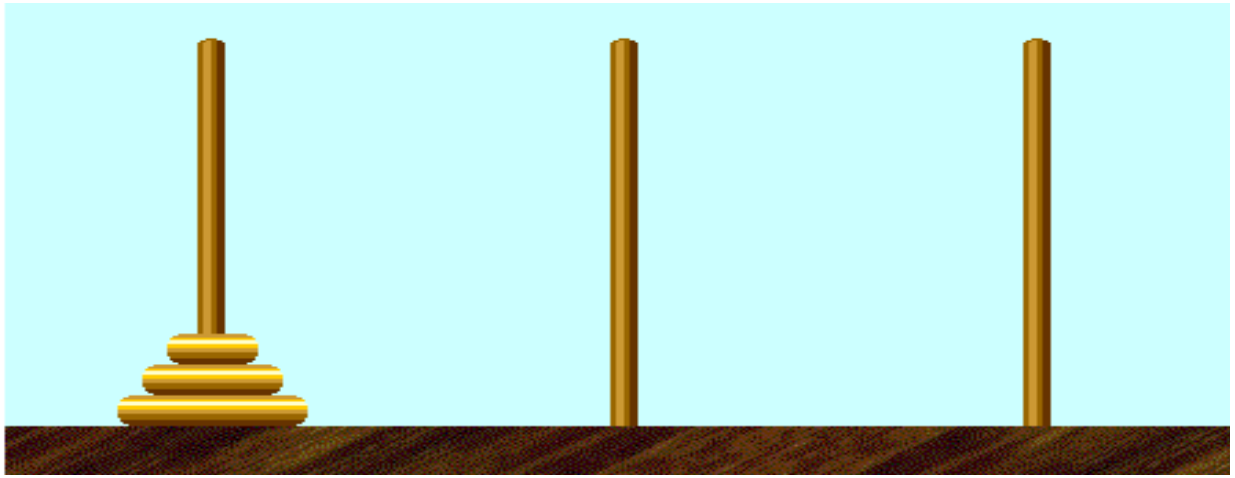
# Exercícios: Outras Funções

isec
Politécnico de Coimbra

1. Calcular a potência de um número inteiro
   - Devolve o valor calculado
2. Verificar se um vetor de inteiros está ordenado de forma crescente
   - Devolve 1 se estiver ordenado (0, caso contrário)
3. Verificar se duas palavras são iguais
   - Devolve 1 se forem iguais (0, caso contrário)

# Torres de Hanói

- Puzzle surgido no final do século XIX, inspirado numa lenda Hindu:
  - 3 torres A, B, C e vários discos de diâmetro diferente



  - Objectivo: Mover todos os discos da torre A para a torre C

# Torres de Hanói: Regras e Restrições

isec
Politécnico de Coimbra

- Regra do jogo:
  - Em cada passo só é possível mover o disco que está no topo de qualquer uma das torres
  - Restrição:
    - Um disco só pode ser colocado numa torre vazia ou sobre outro de diâmetro superior
  - De acordo com a lenda, os monges tinham que resolver um puzzle com 64 discos:
    - Número mínimo de jogadas é igual a $(2^{64}–1)$!!!

# Torres de Hanói: Resolução

isec
Politécnico de Coimbra

- Passo 1:
  - Resolver no papel (ou na animação flash) um problema com 4 discos
- Passo 2:
  - Implementar uma função não-recursiva em C que resolva o problema para qualquer número de discos.
  - A função deve indicar a sucessão de passos até atingir a solução final

# Torres de Hanói: Resolução

isec
Politécnico de Coimbra

- Passo 3:
  - Implementar uma função recursiva em C que resolva o problema para qualquer número de discos.
  - A função deve indicar a sucessão de passos até atingir a solução final

# Torres de Hanói: Função recursiva

isec
Politécnico de Coimbra

- Mover n discos da torre A para a torre C
  - Caso particular: Se n=1 então a solução é trivial
  - Caso geral: A solução para n discos (n≠1) pode ser obtida a partir da solução para n-1 discos.
    - Deslocar os n-1 discos que se encontram em cima de A para B utilizando C como auxiliar
    - Deslocar o disco restante de A para C
    - Deslocar os n-1 discos de B para C utilizando A como auxiliar

# Torres de Hanói: Função recursiva

isec
Politécnico de Coimbra

- Considera-se que:
  - Os discos estão numerados de 1 a n (sendo 1 o menor)
  - As torres são identificadas pelos caracteres A, B, C
  - A função escreve no monitor quais os movimentos efetuados
  - Recebe como argumentos:
    - Número de discos a movimentar
    - Torre inicial
    - Torre final
    - Torre auxiliar

# Torres de Hanói: Função recursiva

```
#include <stdio.h>

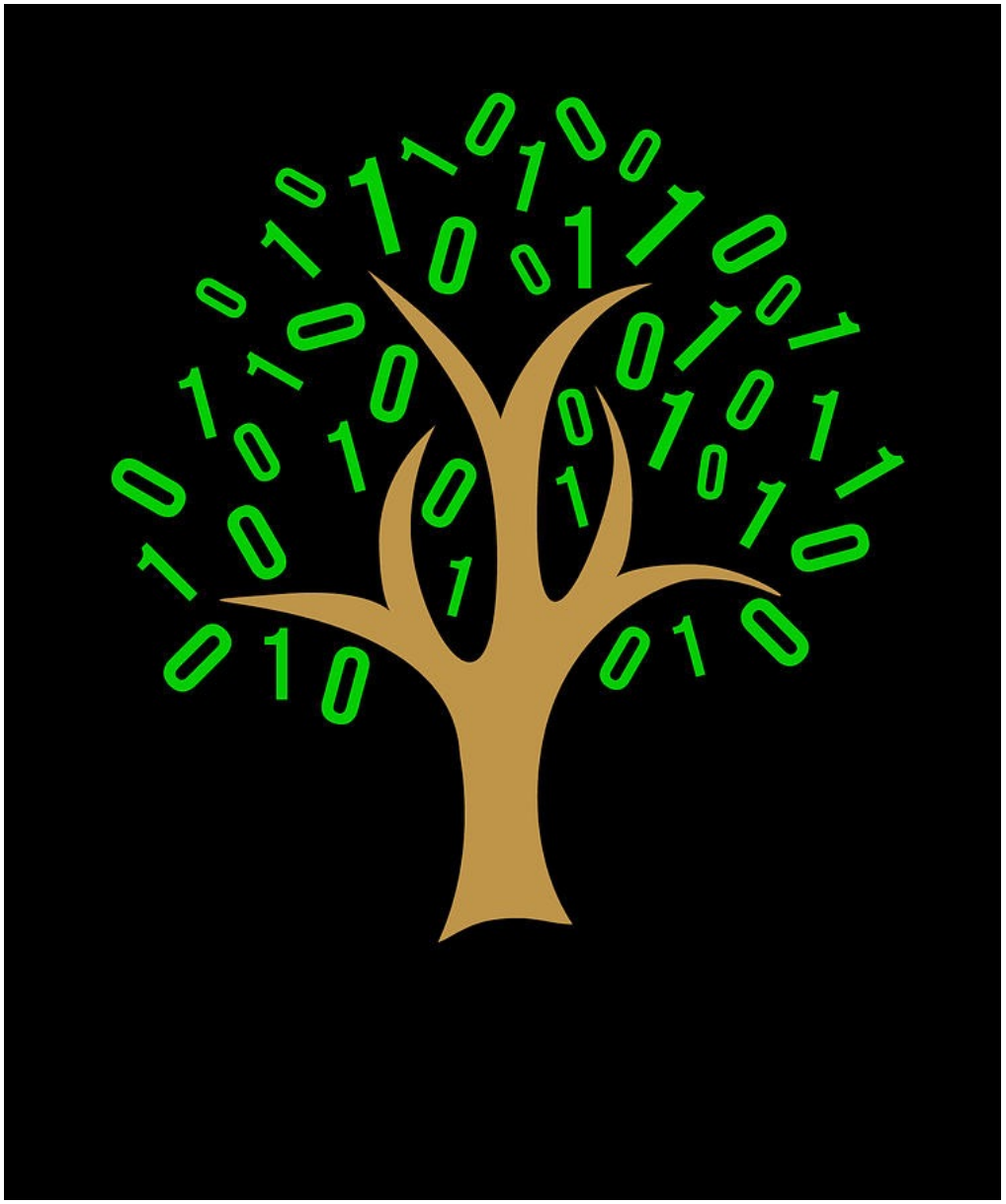
void hanoi(int n, char or, char dest, char aux)
{
   if(n == 1)
       printf("Disco %d: %c -> %c\n", n, or, dest);
   else {
       hanoi(n-1, or, aux, dest);
       printf("Disco %d: %c -> %c\n", n, or, dest);
       hanoi(n-1, aux, dest, or);
   }
}

int main()
{
   int n;

   printf("Numero de discos: ");
   scanf("%d", &n);
   hanoi(n, 'A', 'C', 'B');
   return 0;
}
```

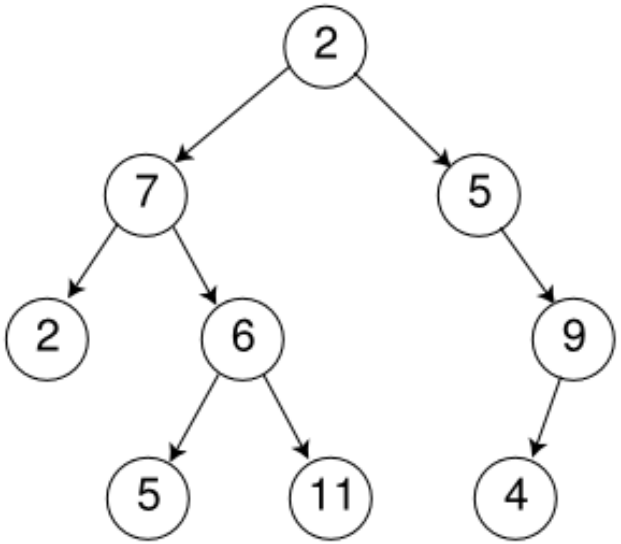
# Árvores Binárias

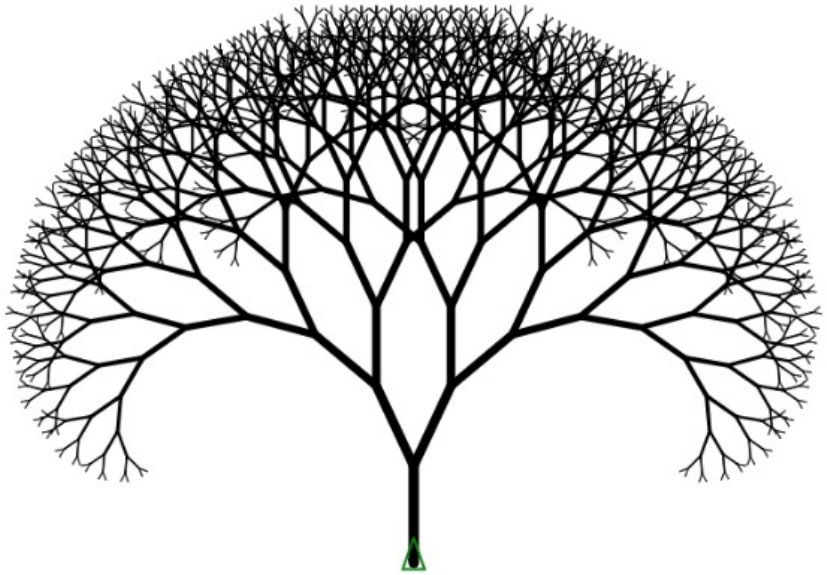
isec
Politécnico de Coimbra



# Árvores binárias

- Estrutura de dados:
  - Nós organizados em formato de árvore
    - 1 raiz
    - Cada nó possui no máximo 2 filhos (esquerdo e direito)
  - Vantagem: eficiência

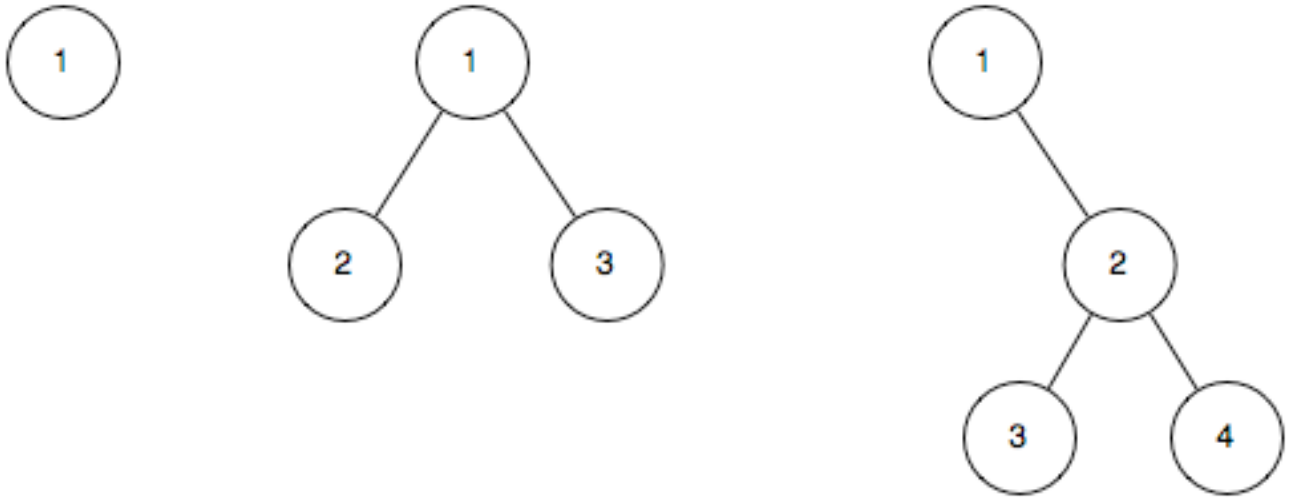


# Árvores binárias: Definição Recursiva

- A árvore está vazia (condição de paragem)

- Ou então, possui um nó (raiz) com 2 filhos:
  - Cada um dos filhos é uma árvore binária



# Árvores binárias

isec
Politécnico de Coimbra

- Alguns termos comuns:
  - Raiz, folha ou nó interno (nó não terminal)
  - Relações ascendente/descendente ou pai/filho
  - Grau de um nó
  - Caminho

# Árvores binárias ordenadas

isec
Politécnico de Coimbra

- Numa árvore binária ordenada os nós estão organizados
  - Para qualquer nó
    - os elementos da sub-árvore esquerda são inferiores
    - os elementos da sub-árvore direita são superiores
  - Manipulação da informação muito eficiente:
    - Pesquisa
    - Inserção

# Árvores binárias

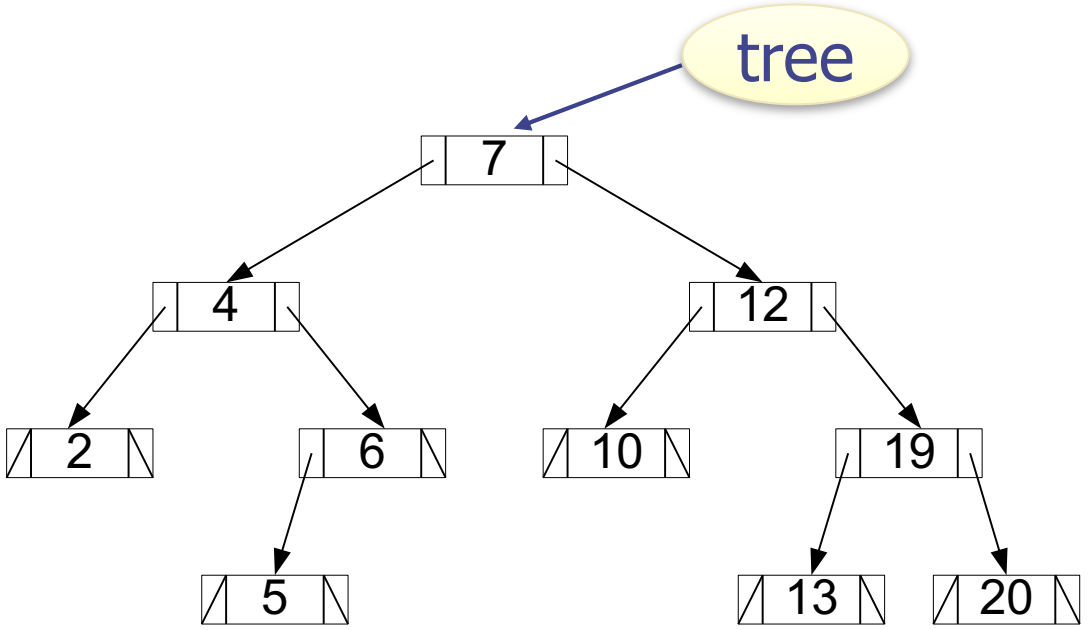
- Implementação como estrutura dinâmica:
  - Existe um ponteiro a referenciar a raiz da árvore
  - Cada nó possui:
    - Informação + 2 ponteiros que referenciam uma sub-árvore esquerda e uma sub-árvore direita
  - NULL representa uma árvore binária vazia

# Árvores binárias: estruturas dinâmicas

```
typedef struct elemento no, *pno;
struct elemento {
   int valor;
   pno esq;
   pno dir;
};
```

```
int main()
{
   pno tree = NULL;
   ...
}
```

NULL: árvore binária vazia



# Processamento de árvores binárias

isec
Politécnico de Coimbra

- Algumas operações simples:
  1. Travessia -> Mostrar toda a informação
  2. Contar o número de nós
  3. Pesquisar um valor numa árvore ordenada
  4. Inserir informação numa árvore ordenada

Operações simples e rápidas

  - Operações de eliminação e reequilíbrio são mais complexas

# Operação 1: Travessia in-order

- Escrever os valores armazenados na árvore referenciada por `tree` de forma ordenada (in-order)

```
void mostra_ordem(pno tree)
{
   if(tree == NULL)
       return;
   else
   {
       mostra_ordem(tree->esq);
       printf("%d\t", tree->valor);
       mostra_ordem(tree->dir);
   }
}
```

# Operação 2: Contar

- Contar o número de nós da árvore referenciada por `tree`
  - Devolve o valor obtido

```
int conta_nos(pno tree)
{
   if(tree == NULL)
       return 0;
   else
       return 1 + conta_nos(tree->esq) +
              conta_nos(tree->dir);
}
```

# Operação 3: Pesquisar

- Procurar o valor *x* na árvore binária ordenada referenciada por `tree`
  - Devolve 1 se encontrar o valor (0, caso contrário)

```
int procura(pno tree, int x)
{
   if(tree == NULL) return 0;
   else if(tree->valor == x) return 1;
   else if(tree->valor > x)
       return procura(tree->esq, x);
   else
       return procura(tree->dir, x);
}
```

# Operação 4: Inserir

- Adicionar um novo nó
  - Adicionar um nó com valor x à árvore binária ordenada referenciada por tree:
    - Devolve referência para raiz da árvore modificada
- O processo vai ser dividido em 2 passos:
  - Criação e preenchimento do novo nó
  - Função recursiva para inserção na árvore

# Operação 4: Inserir

```
// Função recursiva para efectuar a inserção
pno insere_no(pno p, pno novo)
{
   if(p == NULL)
       p = novo;
   else if(p->valor > novo->valor)
       p->esq = insere_no(p->esq, novo);
   else
       p->dir = insere_no(p->dir, novo);
   return p;
}

// Função principal
pno adiciona_no(pno p, int val)
{
   pno novo;

   novo = malloc(sizeof(no));
   if (novo == NULL)
       return p;
   novo->valor = val;
   novo->dir = novo->esq = NULL;
   return insere_no(p, novo);
}
```

# Exercícios: Outras Funções

isec
Politécnico de Coimbra

1. Calcular a altura máxima de uma árvore binária
   - Devolve o valor obtido
2. Calcular a soma de todos os valores armazenados nos nós de uma árvore binária
   - Devolve o total acumulado
3. Calcular o caminho de custo máximo numa árvore binária (entre a raiz e uma das folhas)
   - Devolve o valor do caminho de maior custo